

COMP90038 Algorithms and Complexity

More Divide-and-Conquer Algorithms

Junhao Gan

Week 6 Lecture 12

Semester 1, 2026

Divide and Conquer

We have just discussed the archetypal divide-and-conquer sorting algorithms: MergeSort and (Randomized) QuickSort.

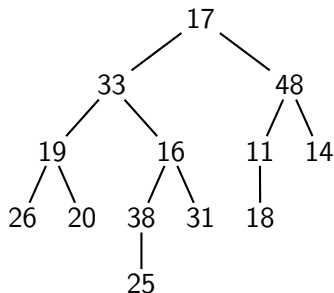
We also introduced the powerful **Master Theorem**, providing solutions to a large class of recurrence relations, for free.

Next, we look at **tree traversal**.

Binary Trees

Binary Tree. A rooted tree T is a binary tree if **every** node in T has **at most 2** child nodes.

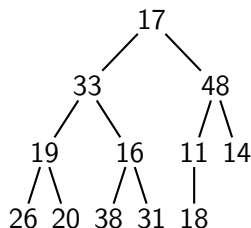
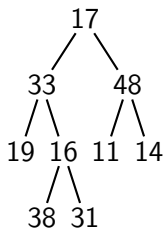
An example of a binary tree:



This tree has **height** 4; a singleton tree has height 0.

Binary Tree Concepts

Special binary trees:



A **full binary tree**: Each node has either 0 or 2 (non-empty) children.

A **complete binary tree**: Each level is filled left to right.

Binary Tree Concepts

Let us call the empty tree *null*.

A non-empty binary tree T has a **root** $T.root$, a **left subtree** $T.left$, and a **right subtree** $T.right$.

Recursion is the natural way of calculating the **height**:

```
function HEIGHT( $T$ )  
  if  $T = null$  then  
    return  $-1$   
  if  $T$  consists of only one node then  
    return  $0$   
  else  
    return  $\max(\text{HEIGHT}(T.left), \text{HEIGHT}(T.right)) + 1$ 
```

Binary Tree Traversal

In general, a binary tree T can be traversed by **recursion**:

- visit $T.root$;
- recursively traverse $T.left$;
- recursively traverse $T.right$.

Depending on when to visit $T.root$, there have three types of traversals:

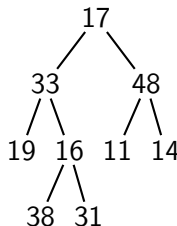
- **Preorder traversal**: visits the root, then the left subtree, and finally the right subtree.
- **Inorder traversal**: visits the left subtree, then the root, and finally the right subtree.
- **Postorder traversal** : visits the left subtree, the right subtree, and finally the root.

In addition, there also exists another type of traversal:

Level-order traversal visits the nodes, level by level, starting from the root.

Binary Tree Traversal: Preorder

```
procedure PREORDERTRAVERSE( $T$ )  
  if  $T \neq \text{null}$  then  
    visit  $T.\text{root}$   
    PREORDERTRAVERSE( $T.\text{left}$ )  
    PREORDERTRAVERSE( $T.\text{right}$ )
```



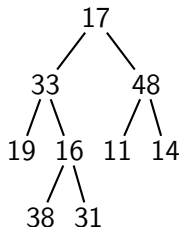
Visit order for the example: 17, 33, 19, 16, 38, 31, 48, 11, 14.

Question: What's the relation between preorder traversal and DFS?

Answer: The preorder traversal is the visiting order (of which elements are pushed into the stack) in DFS.

Binary Tree Traversal: Inorder

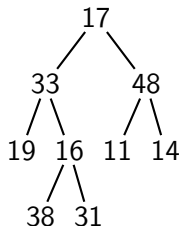
```
procedure INORDERTRAVERSE( $T$ )  
  if  $T \neq \text{null}$  then  
    INORDERTRAVERSE( $T.\text{left}$ )  
    visit  $T.\text{root}$   
    INORDERTRAVERSE( $T.\text{right}$ )
```



Visit order for the example: 19, 33, 38, 16, 31, 17, 11, 48, 14.

Binary Tree Traversal: Postorder

```
procedure POSTORDERTRAVERSE( $T$ )  
  if  $T \neq \text{null}$  then  
    POSTORDERTRAVERSE( $T.\text{left}$ )  
    POSTORDERTRAVERSE( $T.\text{right}$ )  
    visit  $T.\text{root}$ 
```

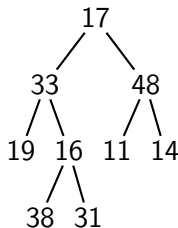


Visit order for the example: 19, 38, 31, 16, 33, 11, 14, 48, 17.

Question: What's the relation between postorder traversal and DFS?

Answer: The postorder traversal is the traversal order (of which elements are popped out of the stack) in DFS.

Binary Tree Traversal: Level-Order



Visit order for the example: 17, 33, 48, 19, 16, 11, 14, 38, 31.

Question: How would you implement the level-order traversal?

Answer: Just BFS.

Non-Examinable Contents

The 2-Dimensional Closest Pair Problem

The Closest Pair Problem (Revisited)

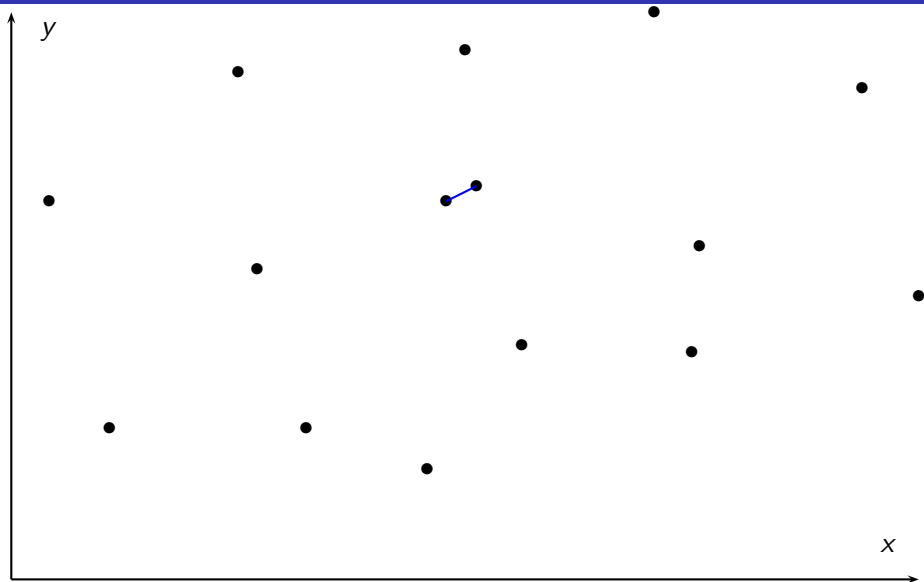
Consider two points $p_1 = (a_1, a_2, \dots, a_d)$ and $p_2 = (b_1, b_2, \dots, b_d)$ in d -dimension space; the *Euclidean Distance* between p_1 and p_2 is computed as:

$$\text{dist}(p_1, p_2) = \sqrt{\sum_{i=1}^d (a_i - b_i)^2}.$$

The 2-Dimensional Closest Pair Problem:

Given a set S of n points in 2-dimensional space, the goal is to find a pair of points p_1^* and p_2^* in S such that $\text{dist}(p_1^*, p_2^*)$ is *no more than* the distance between any other pair of points in S .

The 2D Closest Pair Problem

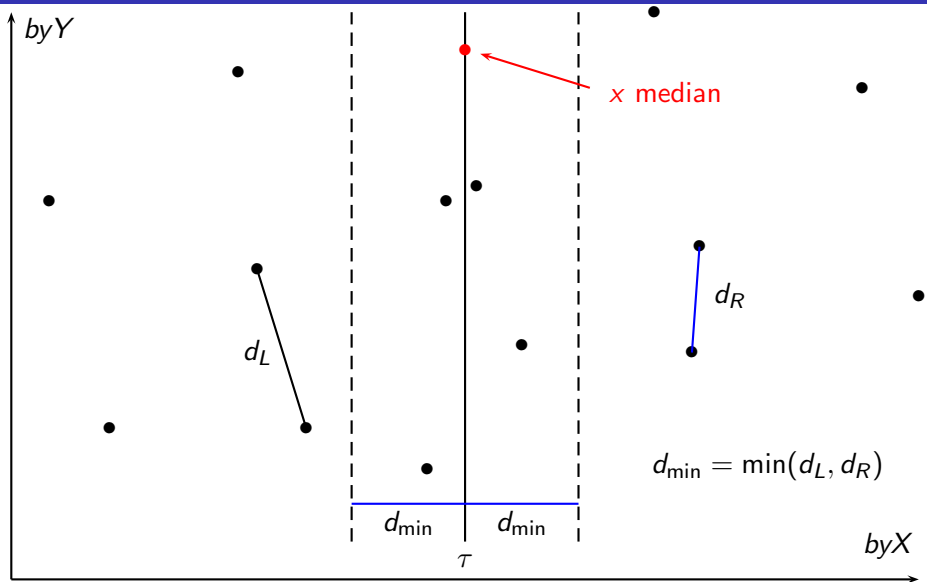


The 2D Closest Pair Problem (Revisited)

In Lecture 5, we gave a brute-force algorithm with worst-case time complexity bounded by $O(n^2)$.

Next, we introduce an improved algorithm which achieves worst-case time complexity of $O(n \cdot \log n)$ by a divide-and-conquer approach.

The Closest Pair Problem Revisited



A New 2D Closest Pair Algorithm: High-Level Idea

The **high-level idea** of the new algorithm is as follows:

- **Partition** S into two roughly equal halves with the **median** τ of the x -coordinates: S_L and S_R , where S_L consists of all the points with x -coordinates $\leq \tau$, and S_R contains the rest.
- **Recursively** find the closest pair in S_L and its distance d_L ;
- **Recursively** find the closest pair in S_R and its distance d_R ;
- Let $d_{\min} = \min\{d_L, d_R\}$ and (p_1^*, p_2^*) be corresponding point pair;
- Check **every** point pair (p_1, p_2) with $p_1 \in S_L$ and $p_2 \in S_R$ that **can possibly** have $\text{dist}(p_1, p_2) \leq d_{\min}$:
 - if $\text{dist}(p_1, p_2) \leq d_{\min}$, update $d_{\min}$ and (p_1^*, p_2^*) accordingly;
- Return $d_{\min}$ and (p_1^*, p_2^*) ;

A New 2D Closest Pair Algorithm: Correctness Analysis

Correctness: The correctness of the above algorithm follows from the fact that the closest pair must consists of points in one of these three possibilities:

- (i) both points are from S_L ,
- (ii) both points are from S_R , and
- (iii) one point from each side.

A New 2D Closest Pair Algorithm: Cost Analysis

Define $C(n)$ as the **worst-case cost**. We have the following recurrence:

$$\begin{aligned} C(n) &= O(1), & \text{for } n \leq 2 \\ C(n) &= 2 \cdot C(n/2) + f(n) + g(n), & \text{for } n > 2 \end{aligned}$$

where:

- $f(n)$ is the cost of dividing the problem into two sub-problems on S_L and S_R , and
- $g(n)$ is the cost of checking all **potential** point pairs across S_L and S_R that can possibly be the closest pair.

We claim that there exists an implementation of the above new algorithm such that $f(n) \in O(n)$ and $g(n) \in O(n)$.

If this is the case, by the Master Theorem, we have $C(n) = O(n \cdot \log n)$.

A New 2D Closest Pair Algorithm: Cost Analysis

Let $L_x(S)$ be the **sorted** list of all the points of S by their **x-coordinates** in a non-decreasing order.

Let $L_y(S)$ be the **sorted** list of all the points of S by their **y-coordinates** in a non-decreasing order.

Claim 1: Given $L_x(S)$ and $L_y(S)$, the closest pair problem on S can be divided into two sub-problems on S_L and S_R with the corresponding sorted lists ready in $O(n)$ time.

A New 2D Closest Pair Algorithm: Cost Analysis

Proof: The algorithm can be implemented as follows:

- Find the *median* value τ in $L_x(S)$ in $O(1)$ time;
- Partition S into S_L and S_R in $O(n)$ time.
- Scan $L_x(S)$ to obtain $L_x(S_L)$ and $L_x(S_R)$ in $O(n)$ time;
- Scan $L_y(S)$ to obtain $L_y(S_L)$ and $L_y(S_R)$ in $O(n)$ time;

Thus, the sorted lists of $L_x(S_L)$ and $L_y(S_L)$ are ready for the recursion on S_L . The same applies to S_R .

Therefore, the problem dividing cost $f(n) \in O(n)$.

A New 2D Closest Pair Algorithm: Cost Analysis

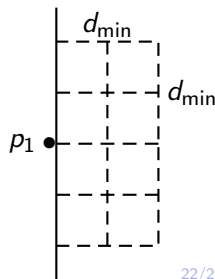
Claim 2: Given the minimum distance $d_{\min} = \{d_L, d_R\}$, there can be *at most* $8 \cdot n$ possible point pairs (p_1, p_2) with $p_1 \in S_L$ and $p_2 \in S_R$ that *can possibly* have $\text{dist}(p_1, p_2) \leq d_{\min}$.

Proof: All the candidate point pairs (p_1, p_2) must lie in the *strip* with x -coordinates between $\tau - d_{\min}$ to $\tau + d_{\min}$.

Moreover, consider a point $p_1 \in S_L$ in the strip; all possible candidate points $p_2 \in S_R$ with respect to p_1 must satisfy:

- $|p_1.y - p_2.y| \leq d_{\min}$, and
- all these p_2 's must have distance $\geq d_{\min}$ from each other.

As shown in the figure, each small cell can contain *at most* one p_2 . (**Why?**) Therefore, each $p_1 \in S_L$ in the strip can have *at most* 8 points $p_2 \in S_R$ in the strip which can possibly have $\text{dist}(p_1, p_2) \leq d_{\min}$.



A New 2D Closest Pair Algorithm: Cost Analysis

Claim 3: Given $L_y(S)$ and $d_{\min}$, all the point pairs (p_1, p_2) with $p_1 \in S_L$ and $p_2 \in S_R$ which can possibly have $\text{dist}(p_1, p_2) \leq d_{\min}$ can be checked in $O(n)$ time, i.e., $g(n) \in O(n)$.

Proof: By scanning $L_y(S)$, we can obtain the list L_M of all the points having x-coordinates in $[\tau - d_{\min}, \tau + d_{\min}]$, and the points in L_M are sorted by their y-coordinates in a non-decreasing order. Clearly, this can be done in $O(n)$ time.

Scan every point p in L_M ; if $p \in S_L$, check all the points q in L_M with $|q.y - p.y| \leq d_{\min}$ by scanning L_M from p in both directions:

- if $q \in S_L$, skip q ;
- if $q \in S_R$ and $\text{dist}(p, q) < d_{\min}$, then update $d_{\min}$ and record the pair (p, q) ;

A New 2D Closest Pair Algorithm: Cost Analysis

Proof of Claim 3: (Continue)

By Claim 2 and a similar argument, we know that there can be at most 16 such points q 's to check, where **eight** from S_L and **eight** from S_R .

Therefore, the overall checking cost for all those point pairs is bounded by $O(n)$, and thus, $g(n) = O(n)$.

A New 2D Closest Pair Algorithm

By Claims 1 - 3 and the previous cost analysis, given $L_x(S)$ and $L_y(S)$, we conclude that the worst-case overall running time of our new 2D Closest Pair Algorithm is bounded by $O(n \cdot \log n)$.

But, wait... Something is missing...

How do we obtain $L_x(S)$ and $L_y(S)$?

We can sort the points in S by their x -coordinates and y -coordinates, respectively, with MergeSort. The cost for sorting is bounded by $O(n \cdot \log n)$.

The overall running time bound of our 2D Closest Pair Algorithm remains the same.

Exercise: Describe the full 2D closest pair algorithm.

The detailed implementation description can be exacted from the above discussions.